

Table Attribute in Entity Framework Core

In this article, I will discuss **Table Data Annotation Attribute in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing the basics of [Data Annotation Attributes in Entity Framework Core](#).

What is Table Attribute in Entity Framework Core?

The Table attribute in Entity Framework Core (EF Core) explicitly specifies the database table name and optionally specifies the schema for a domain class. This attribute is part of the `System.ComponentModel.DataAnnotations.Schema` namespace and allows us to override the default naming conventions that EF Core follows when creating or mapping tables.

Default Table Name Convention in EF Core

Without the Table attribute, EF Core uses one of the following conventions to determine the table name:

- **Entity Class Name:** By default, EF Core names the table after the entity class name (e.g., a `Student` class maps to a `Student` table).
- **DbSet<T> Property Name:** If the entity is exposed as a `DbSet<T>` property in the `DbContext` class and the property name differs from the entity class name, the property name takes precedence.

For example, If a `DbSet<Student>` is defined as `public DbSet<Student> StudentsInfo { get; set; }`, the table name defaults to `StudentsInfo`.

Usage of the Table Attribute in EF Core

The `[Table]` attribute allows us to customize the table name and schema to which an entity is mapped. This is especially useful when:

- **Custom Table Names:** The default table name generated by EF Core does not meet our requirements.
- **Mapping to Existing Databases:** We are working with an existing database and need to map the entity to a specific table name or schema.

Definition of Table Attribute Class in Entity Framework Core

All Data Annotation attributes, including the Table attribute, inherit from the abstract class `Attribute`. If you look at the definition of the `TableAttribute` class, you will see the following:

```

namespace System.ComponentModel.DataAnnotations.Schema
{
    ... public class TableAttribute : Attribute
    {
        private string? _schema;

        ... public TableAttribute(string name) ...

        ... public string Name ...
        ... public string? Schema ...
    }
}

```

The TableAttribute includes the following two properties:

- **Name (Mandatory):** This property is set through the constructor when applying the [Table] attribute to a class. Since there is no setter provided outside the constructor, it is a read-only property, and we must specify the name of the table when creating an instance of the TableAttribute class.
- **Schema (Optional):** This property has both getter and setter accessors, making it a read-write property. If not specified, the default schema used is dbo.

Examples to understand Table Data Annotation Attribute in EF Core:

Let us understand the Table attribute in Entity Framework Core with an example. In the example below, we modify the Student entity class to specify the table name as StudentInfo. EF Core will create or map the Student entity to the StudentInfo table in the default dbo schema.

```

using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    [Table("StudentInfo")] // Mapping the entity to StudentInfo table in dbo
    schema
    public class Student
    {
        public int StudentId { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
    }
}

```

Note: In the above example, the [Table("StudentInfo")] attribute instructs EF Core to create or map the Student entity to the StudentInfo table under the default dbo schema.

DbContext Configuration

Ensure the DbContext includes a DbSet property for the Student entity. So, please modify the context class as follows. As you can see, we only include the Student as a DbSet Property.

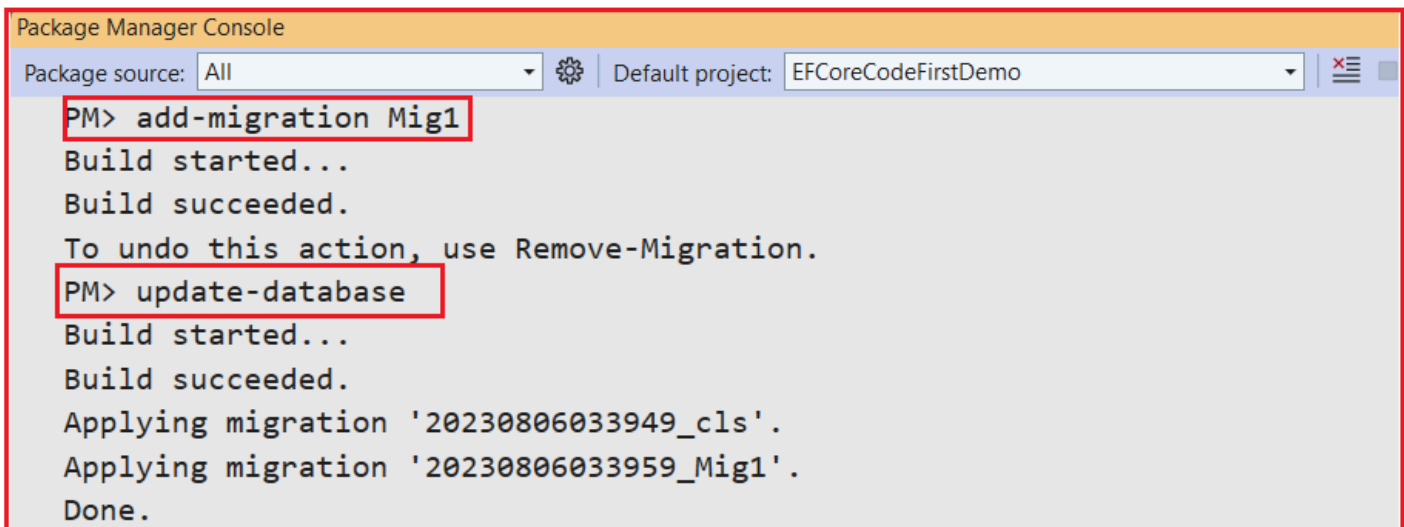
```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        public DbSet<Student> Students { get; set; }
    }
}

```

Migration Process:

After modifying the entity classes and DbContext, we need to create and apply migrations to update the database schema. So, open the Package Manager Console and Execute the following **Add-Migration** and **Update-Database** commands. You can give your migration any name. Here, I am giving Mig1. The name you are giving should not be given earlier.



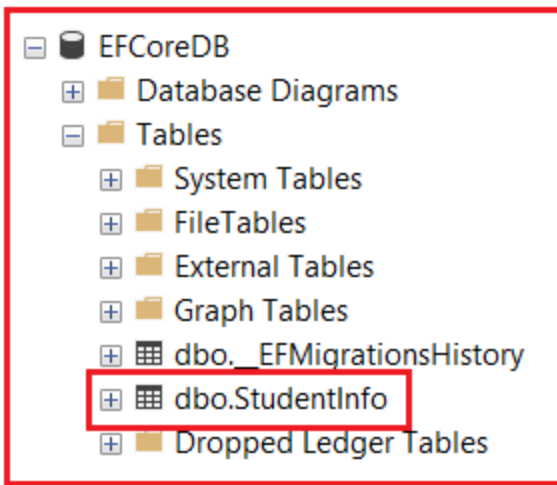
The screenshot shows the Package Manager Console window for the project 'EFCoreCodeFirstDemo'. The package source is set to 'All'. The console output shows the following commands and their results:

```

PM> add-migration Mig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20230806033949_cls'.
Applying migration '20230806033959_Mig1'.
Done.

```

After executing these commands, you can verify that EF Core created the StudentInfo table with the default dbo schema, as shown in the image below.

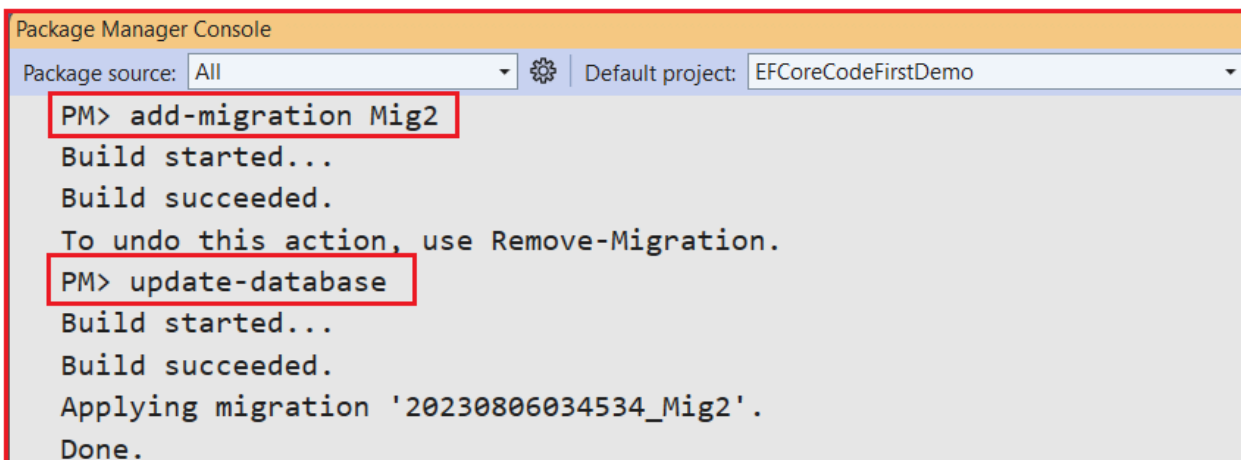


Specifying a Different Schema:

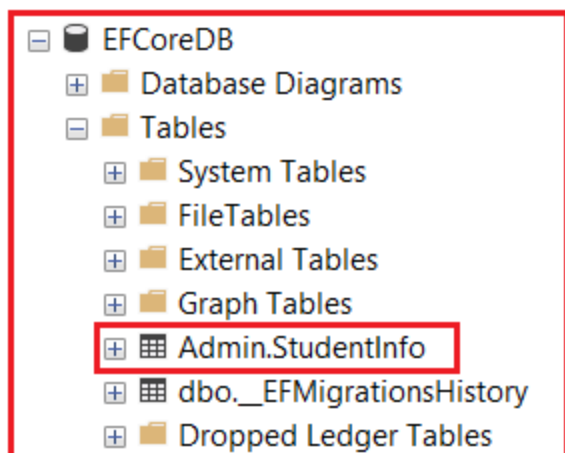
If you want to create a table with a different schema, such as Admin, you need to use the Schema property of the Table attribute. To better understand, please modify the Student entity as follows.

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    [Table("StudentInfo", Schema = "Admin")] // Mapping the entity to the
    StudentInfo table in Admin schema
    public class Student
    {
        public int StudentId { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
    }
}
```

Again, open the Package Manager Console and Execute the following **add-migration** and **update-database** commands. Whenever we change our model or DbContext, we must execute the following two commands. I cannot use Mig1, so I am giving the migration name Mig2.



After running the migration and updating the database, the table StudentInfo will be created under the Admin schema. Now, you can verify the database, and you should see the schema as Admin and the table name as StudentInfo, as shown in the image below.



When Should We Use Table Attribute in EF Core?

The [Table] attribute is particularly useful in the following scenarios:

- **Custom Naming Conventions:** When your database naming conventions differ from your entity class naming conventions, the [Table] attribute allows you to align them without renaming your entity classes.
- **Schema Specification:** When you need to map entities to specific schemas other than the default dbo. This is especially useful when working with a database that requires organizing tables into different schemas because of the security.
- **Existing Databases:** When working with an existing database, the [Table] attribute ensures that your domain classes correctly map to the pre-defined table names and schemas.

In the next article, I will discuss [Column Data Annotation Attribute in Entity Framework Core](#) with Examples. In this article, I explain **Table Data Annotation Attribute in Entity Framework Core** with examples. I hope you enjoyed this Table Attribute in EF Core with Examples article. Please provide your valuable feedback and suggestions for this article.